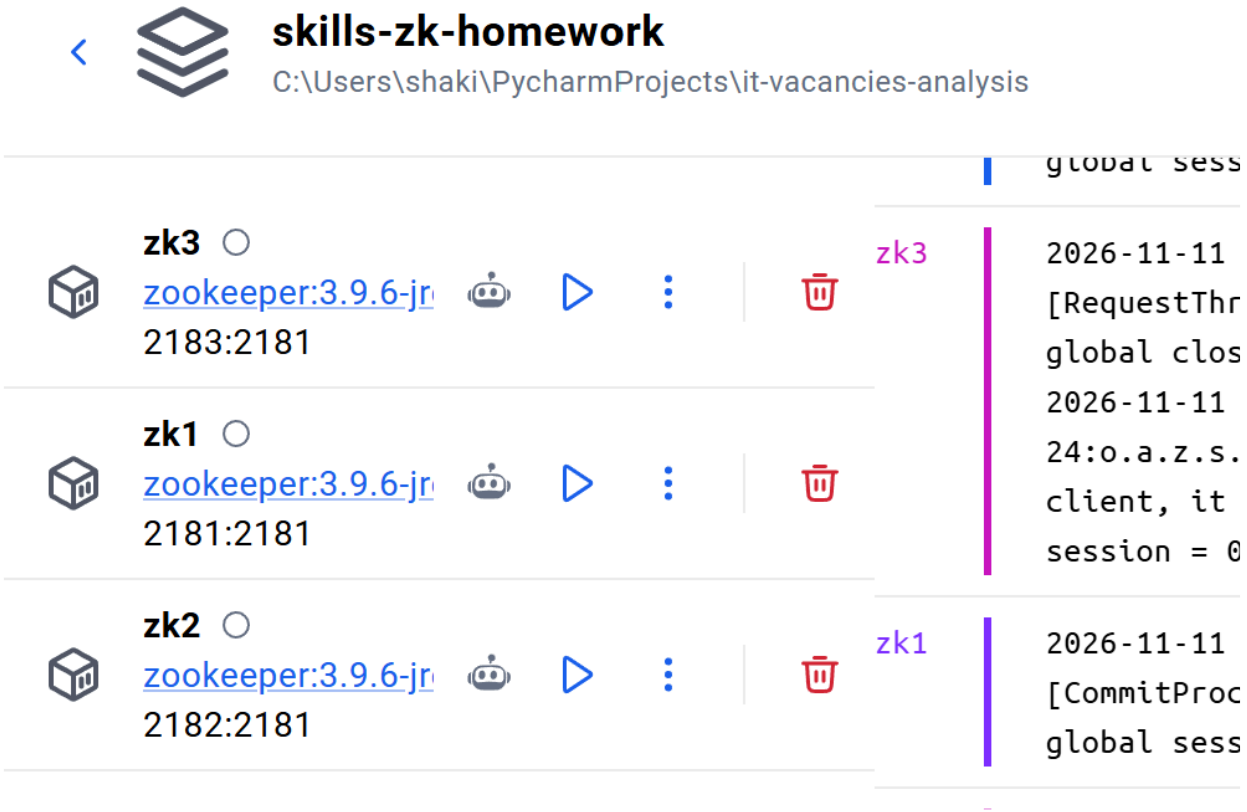
Отказоустойчивость средствами Apache ZooKeeper

Установка и подготовка

Выбран предусмотренный заданием Docker-вариант: три отдельных сервера. Java находится внутри каждого контейнера.



Ансамбль ZooKeeper

Конфигурации, порты, каталоги данных и myid

Сервер	myid	Порт на компьютере	Том данных	Том журнала транзакций
zk1	1	2181	zk1-data	zk1-log
zk2	2	2182	zk2-data	zk2-log
zk3	3	2183	zk3-data	zk3-log

```
    ZOO_SERVERS: server.1=zk1:2888:3888;2181 server.2=zk2:2888:3888;2181 server.3=zk3:2888:3888;218
    ZOO_TICK_TIME: 2000
    ZOO_INIT_LIMIT: 10
    ZOO_SYNC_LIMIT: 5
    ZOO_4LW_COMMANDS_WHITELIST: srvr,ruok,mntr,stat
    ZOO_ADMINSERVER_ENABLED: "false"
services:
  zk1:
    <<: *zookeeper
    hostname: zk1
    environment:
      <<: *environment
      ZOO_MY_ID: 1
    ports: ["127.0.0.1:2181:2181"]
    volumes: ["zk1-data:/data", "zk1-log:/datalog"]
  zk2:
    <<: *zookeeper
    hostname: zk2
    environment:
      <<: *environment
      ZOO_MY_ID: 2
    ports: ["127.0.0.1:2182:2181"]
```

Роли получены диагностической командой `srvr`: первоначально zk2 - лидер, zk1 и zk3 - последователи. Лидер ансамбля ZooKeeper и лидер координаторов расчёта выбираются отдельно.
Начальные роли трёх серверов ансамбля:

```
"initial": {
  "zk1": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UTC\nLaten
  "zk2": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UTC\nLaten
  "zk3": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UTC\nLaten
```

```
tions: 1\nOutstanding: 0\nZxid: 0x100000001\nMode: follower\nNode count: 5\n",
tions: 1\nOutstanding: 0\nZxid: 0x100000001\nMode: leader\nNode count: 5\nProposal sizes last/min/max: 48/48/48\n",
nections: 2\nOutstanding: 0\nZxid: 0x100000001\nMode: follower\nNode count: 5\n"
```

Остановка одного, затем двух серверов и восстановление

Ответы команды `srvr` и результаты клиентских операций:
ensemble.json.

Состояние	zk1	zk2	zk3	Проверка записи
Три сервера	follower	leader	follower	Успешное создание узла
Остановлен zk1	остановлен	leader	follower	Записано и прочитано `two alive`
Остановлены zk1 и zk3	остановлен	не обслуживает запросы	остановлен	`KazooTimeoutError`
Серверы возвращены	follower	follower	leader	Записано и прочитано `restored quorum`

После остановки двух серверов живой zk2 ответил:

```
This ZooKeeper instance is not currently serving requests
```

Запись из существующей сессии не завершилась за отведённое время; новое подключение также завершилось `KazooTimeoutError`. Тайм-аут записи означает неизвестный итог операции после восстановления соединения, после восстановления проверена новая успешная запись.
После остановки zk1 два оставшихся сервера продолжают принимать запись:

```
"one_stopped": {
  "zk1": "timed out",
  "zk2": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 20
  "zk3": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 20
},
"write_with_two": "two alive",
```


При двух отказах кворума нет; после возврата серверов запись восстановлена

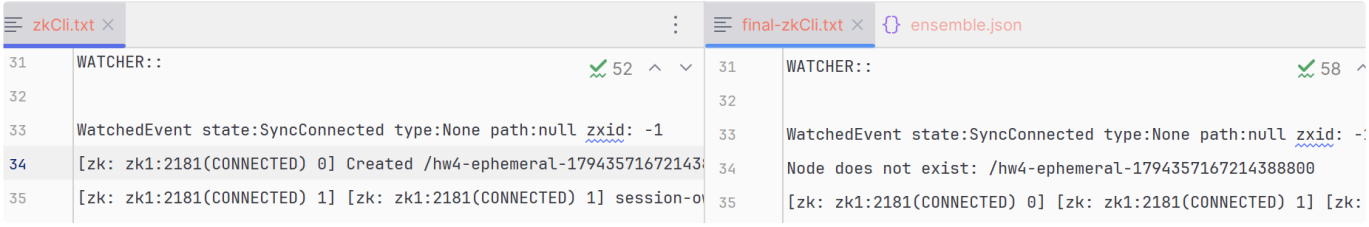

```
"two_stopped": {
  "zk1": "timed out",
  "zk2": "This ZooKeeper instance is not currently serving requests\n",
  "zk3": "timed out"
},
"no_quorum_write_error": "KazooTimeoutError",
"no_quorum_connect_error": "KazooTimeoutError",
"restored": {
  "zk1": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UT
  "zk2": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UT
  "zk3": "Zookeeper version: 3.9.6-a355171b081b5b60749db8f19cca1528b0df936f, built on 2026-09-03 19:29 UT
},
"write_after_restore": "restored quorum",
```


Создание эфемерного узла

Выполнено:

```
create -e /hw4-ephemeral-1794357167214388800 session-owned
get -s /hw4-ephemeral-1794357167214388800
quit
```

До закрытия сессии: `ephemeralOwner = 0x10001c6931d0000`, данные `session-owned`.
После `quit` сессия закрылась, независимый клиент получил отсутствие узла (`null`).
Полный вывод: zkCli.txt.
Дополнительное чтение отсутствующего узла и итоговых указателей: final-zkCli.txt.
Эфемерный узел существовал до закрытия сессии и исчез после её закрытия:



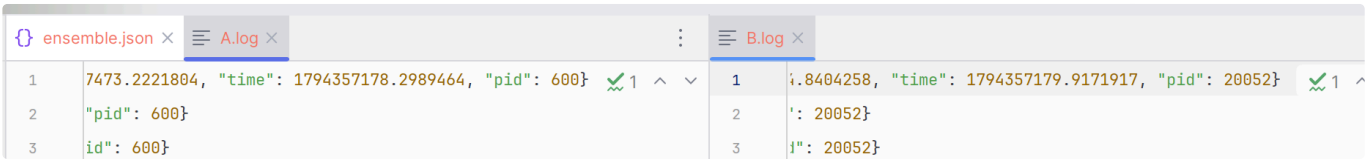
Приостановка лидера без защиты

Предыдущий расчёт использовал MapReduce и сравнение с DuckDB. В ДЗ 4 добавлен координатор, использующий исходные `map_chunk` и `save_counts`.
Сохраняется исходное условие: роль «Программист, разработчик», C/C++ в названии, исходная нормализация, подсчёт наборов размера 1, 2, 3, включая требуемые пары.
Два отдельных процесса A и B используют `Kazoo Lock`: эфемерные последовательные узлы и наблюдение за предшественником. Победитель атомарно увеличивает `/epoch`.
Чекпоинт содержит обработанное количество вакансий, накопленные частоты и хеш входа.
Неизменяемый файл закрывается до публикации указателя на него; при восстановлении проверяются SHA-256 файла и принадлежность входному набору.
Публикация чекпоинта в ZooKeeper всегда проверяет эпоху транзакцией, поэтому старый процесс не может откатить прогресс нового лидера даже в сценарии без защиты результата.

Запуск без защиты:

```
03:33:22.518 B: записал результат, version=1, epoch=2
03:33:48.756 A: записал результат, version=2, epoch=1
```

Журналы: A.log, B.log, summary.json.
Два координатора с сессиями 10 секунд; запасной продолжил с чекпоинта 600/3676



Без защиты устаревший лидер повторно записал результат; смена лидера - 11,428 с

```
"backup_restored_offset": 600,
"publications": 2,
"pointer_version": 2,
"pointer": {
```

Совпадение с ДЗ 3

В обоих процессах получены 3676 вакансий, 17565 уникальных пар навыков и 59034 вхождения пар. Все три CSV каждого процесса совпали с эталоном ДЗ 3.
Полные контрольные числа и хеши приведены ниже.

Приостановка лидера с защитой по эпохе

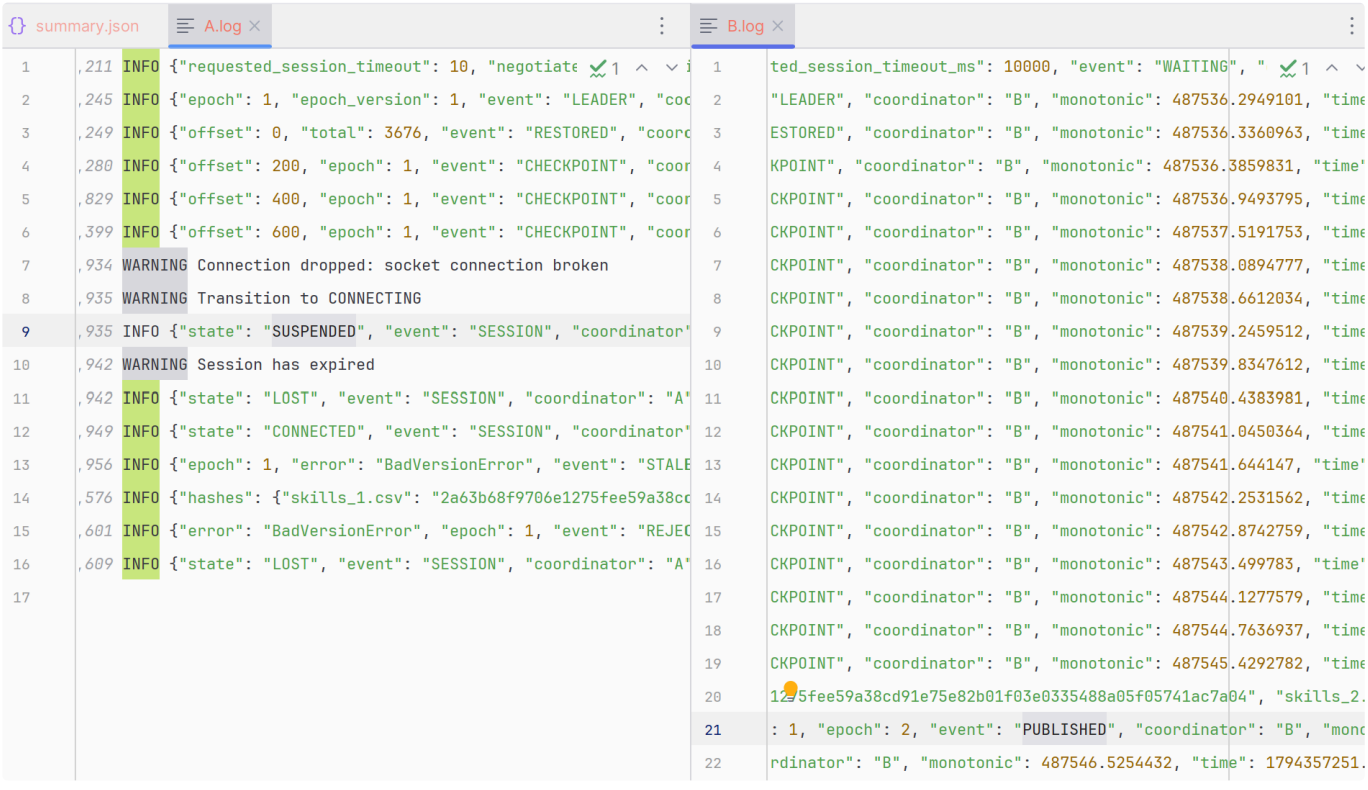
Повторение эксперимента с FENCING=1

Повторён тот же сценарий: два отдельных координатора, сессии 10 секунд, приостановка A примерно на 40 секунд после чекпоинта и завершение расчёта B до возобновления A. Проверка финальной публикации включена параметром `--fencing 1`, эквивалентным `FENCING=1`. B восстановил offset 600, стал лидером через 10,463115 с и опубликовал результат с эпохой 2.

Отклонение устаревшей записи

С защитой, запуск `fencing-1-1794357228831722700`: B публикует результат эпохи 2; при попытке A опубликовать результат эпохи 1 транзакция отклоняется с `BadVersionError`.
Журналы: A.log, B.log, summary.json.
Указатель создаётся пустым с версией 0; каждая принятая публикация увеличивает версию. Файлы кандидата A создаются и при включённой защите, но они не становятся опубликованным
результатом: итоговый указатель остаётся на B.

Защита отклонила публикацию A; единственный опубликованный результат принадлежит B:



Сравнение двух сценариев

Показатель	Без защиты (`FENCING=0`)	С защитой (`FENCING=1`)
Время от паузы A до события LEADER у B	**11,4282404 с**	**10,4631150 с**
B восстановил чекпоинт	600 / 3676	600 / 3676
Успешных публикаций результата	**2**	**1**
Итоговая версия `/result`	**2**	**1**
Итоговый указатель	A, эпоха 1 (устаревший)	B, эпоха 2
Публикация возобновившегося A	принята	**BadVersionError**
CSV совпали с ДЗ 3	да	да

Исходные значения и итоговые указатели: verification.json
10 секунд - тайм-аут сессии, а не точное время переключения

Контрольные числа обоих запусков

Размер набора	Уникальных наборов	Сумма частот
1	1647	16218
2	17565	59034
3	86189	200933

Дополнительно проверено полное побайтовое совпадение всех трёх CSV, а не только контрольных сумм частот. SHA-256 эталона и каждого результата A/B в обоих запусках:

```
skills_1.csv  2a63b68f9706e1275fee59a38cd91e75e82b01f03e0335488a05f05741ac7a04
skills_2.csv  afe6b03a13d3a72c07fe208e887afaf83cde10261367e13e4ec03a5f9c7fe3a8
skills_3.csv  22b2c5c90c59c9a4579e0cf515befddb7180541e7ff4980321f8700f47cf0683
```

Эталон: mapper_failure_check.json,
контрольные числа ДЗ 3: mapper_retry_log.txt
Все результаты новых запусков: results/zookeeper

Почему три сервера, а не два и не четыре?

Для принятия решений нужна большая половина серверов. Любые два большинства пересекаются, поэтому две части ансамбля не могут одновременно принимать независимые записи. У двух серверов большинство - два: потеря любого останавливает запись. У трёх большинство - два: один сервер может отказать.
У четырёх большинство - три: допустим всё ещё только один отказ, хотя серверов и обменов стало больше. Поэтому три - минимальное число голосующих серверов для сохранения работы после одного отказа. Для двух отказов нужны пять.

Почему эпоху проверяет получатель записи, а не только ZooKeeper?

После паузы старый процесс может продолжить работу с уже устаревшим знанием «я лидер».
ZooKeeper удалит его эфемерный узел, но не остановит процесс и не запретит запись в сторонний файл, БД или API. Даже проверка лидерства перед отправкой оставляет промежуток: между проверкой и записью лидер может смениться.
Поэтому получатель должен отвергать старые эпохи в той же атомарной операции, в которой принимает результат.
В моей работе получатель — функция `receive_result` ,
а опубликованный ресурс - указатель `/result` в ZooKeeper. Она выполняет `check` версии `/epoch` и `set_data /result` в одной транзакции. У A сохранена версия эпохи 1, после избрания B текущая версия 2; `check` возвращает `BadVersionError` , и указатель вообще не изменяется. Здесь механизм проверки выполняет ZooKeeper, потому что сам ресурс расположен в нём. Для результата в отдельной БД проверять эпоху и менять результат должна сама БД в одной транзакции; отдельный предварительный запрос к ZooKeeper такую гарантию не даёт.